



GSTACK CASE SAMPLES

GStack, Beyond Office Hours

Real, worked scenarios for the other 22 skills — from locking architecture to shipping and reflecting on a live product.

Scenarios drawn from a React + Vite + Tailwind marketing-site build — the kind of work behind projects like AgriMind AI.

Garry Tan's open-source Claude Code stack · MIT licensed · Free forever

One toolkit, seven roles on your team



23 skills + 8 power tools

A CEO, a designer, an eng manager, a QA lead, a security officer, and a release engineer — all slash commands, all Markdown, all free.



One sprint structure

Think → Plan → Build → Review → Test → Ship → Reflect. Each skill writes an artifact the next one reads.



Works past Claude Code

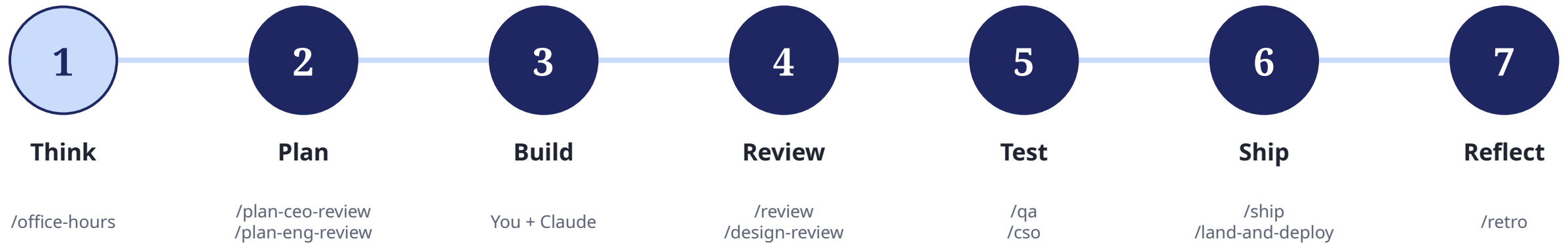
Ten agents supported — Codex, Cursor, OpenClaw, Factory, Kiro, and more — with the same skill set.



Free and open

MIT licensed, no premium tier, no waitlist. Fork it, edit any workflow, make it yours.

Office hours is step one of seven



Office hours writes the design doc every downstream skill reads. This deck covers the six stages after it — with scenarios from a React + Vite + Tailwind marketing-site build, the kind of work behind AgriMind AI.



PLAN

/plan-ceo-review

Find the 10-star product hiding in the request

THE ASK

“Add a contact form to the marketing site.”

A one-line feature request — the kind that ships in an afternoon and quietly under-delivers.

WHAT THE SKILL DOES

Runs four scope modes — Expansion, Selective Expansion, Hold Scope, Reduction — and pushes back on the framing before anyone writes code.

For a contact form on an agtech site, that means asking who actually fills it out — and reframing it as a qualified-lead intake wired to the existing email integration, so a bulk buyer routes differently from a newsletter signup.



PLAN

/plan-eng-review

Lock the architecture before a single component is built



Data flow, decided up front

Maps how design-system tokens — colors, spacing, the light/dark theme — flow from the Tailwind config into every animated component, before the first one exists.



Breakpoints as an explicit edge case

Forces a decision on what happens at the mobile-to-tablet boundary while the desktop layout stays untouched, instead of discovering the conflict mid-build.



A test plan, written before code

Produces the test matrix that /qa executes later — so testing is planned, not improvised after something breaks.



PLAN → BUILD

/design-shotgun → /design-html

Explore visually, then ship what you picked



EXPLORE

Describe a hero section once. Get 4–6 AI mockup variants, opened side by side in a comparison board.

Leave feedback in plain language — “more whitespace,” “bolder CTA,” “lose the gradient” — and it generates another round. Taste memory learns what you actually pick.



SHIP IT

The mockup you approved becomes production HTML — not a demo. Text reflows, heights adjust, the layout stays dynamic across viewports.

30KB overhead, zero dependencies. Detects React, Svelte, or Vue and matches your framework — so a picked hero mockup lands as real Tailwind markup, not a screenshot to reverse-engineer.



REVIEW

/review

Catches what passes CI but breaks in the browser

AUTO-FIXED

overflow-x: hidden vs. position: sticky

Symptom: the navbar quietly stops sticking on scroll — no error, no failed test, just wrong behavior on real pages.

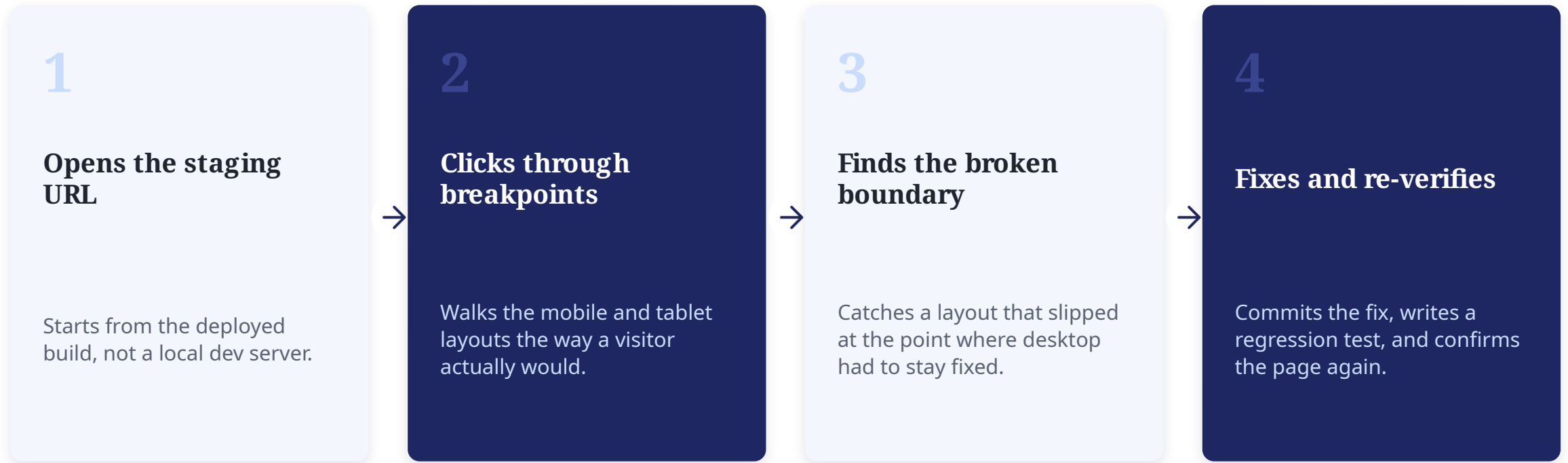
AUTO-FIXED

Tailwind opacity modifiers need RGB-channel vars

Symptom: opacity utility classes render fully solid because the underlying CSS variable isn't in the RGB-channel format Tailwind expects.

Auto-fixes the obvious ones. Flags completeness gaps and anything riskier for your call — it never silently rewrites logic it isn't sure about.

Opens a real browser and finds what a diff can't show



Every /qa fix generates its own regression test — the same bug can't come back unnoticed.

A security review before anyone else finds the gap



OWASP Top 10 + STRIDE

Runs the same threat-modeling checklist a dedicated security review would.



Checks the form-submission endpoint

On a public lead-intake form, that means scanning for exposed keys and injection risk before launch — not after a report comes in.



Zero-noise by design

17 false-positive exclusions and an 8-out-of-10 confidence gate, so findings are worth your time.



Every finding has a scenario

Not a vague warning — a concrete exploit path showing how the gap could actually be used.



SHIP

/ship → /land-and-deploy

From an approved plan to verified in production



/ship syncs main, bootstraps a test framework if the project doesn't have one, audits coverage, then pushes and opens the PR. /land-and-deploy takes it from "approved" to merged, watches CI and the deploy, then checks production health — the same sequence whether the target is Vercel or Railway.



SHIP

/document-release

The technical writer you never had to hire



Reads every doc, cross-references the diff

Compares what changed in the shipped code against what the docs currently claim.



Updates what drifted

README, deployment notes for Vercel and Railway, and brand-asset usage guides — kept current automatically instead of going stale one release at a time.



Runs itself

Auto-invoked at the end of every /ship, so keeping docs current never becomes an extra command to remember.

A weekly retro without the status meeting



Per-person breakdowns

Who shipped what, across every project.



Shipping streaks

Momentum made visible, not just felt.



Test health trends

Coverage direction over time, not a single snapshot.



Growth opportunities

Where the next sprint's effort would pay off most.

Run it as /retro global to see the same view across every project and every AI tool in use — Claude Code, Codex, Gemini — in one place.



REFLECT → SHARE

/diagram → /make-pdf

Turn decisions into artifacts you can hand someone



/diagram

English description in. Out comes a triplet: Mermaid source, an editable .excalidraw file, and a rendered SVG/PNG — all offline, no network calls.

Turns the /plan-eng-review architecture notes — data flow, the theme-token pipeline — into a diagram a co-founder can actually open and edit.



/make-pdf

Markdown in, publication-quality document out. Mermaid and Excalidraw fences render as vector diagrams; wide ones get their own landscape page.

One command turns a retro summary or a design-system writeup into a PDF, HTML page, or Word doc — ready to send to an investor or teammate, not just read on screen.

Which review should I use?

Building for...	Plan stage (before code)	Live audit (after shipping)
End users (UI, web app, mobile)	<code>/plan-design-review</code>	<code>/design-review</code>
Developers (API, CLI, SDK, docs)	<code>/plan-devex-review</code>	<code>/devex-review</code>
Architecture (data flow, perf, tests)	<code>/plan-eng-review</code>	<code>/review</code>
All of the above	<code>/autoplan</code> — runs CEO → design → DX → eng, auto-detecting which apply	—



Not sure which applies? `/autoplan` runs the full pipeline and only surfaces the taste decisions that need your call — eng review always runs last, so it gates the final amended plan.

TRY IT ON YOUR NEXT FEATURE

Six commands, start to finish

- 1** **Install**
30 seconds — one clone command
- 2** **/plan-ceo-review**
On any feature idea
- 3** **/review**
On any branch with changes
- 4** **/qa**
Point it at your staging URL

Stop there — you'll know if this is for you.

Garry Tan's open-source Claude Code stack



FREE & OPEN

MIT licensed. No premium tier, no waitlist. Fork it, edit any workflow, make it yours.

github.com/garrytan/gstack

Also works with Codex, Cursor, OpenClaw, and 6 other agents.